



Implementing Power-Down Sleep Mode on the ATmega328P Microcontroller

Technical Report

Prepared by: Eyob Bantayehu (CTC-2790-26)

Submitted to: Gutenberg

September, 2026

1. Introduction

Battery-powered and always-idle embedded systems spend most of their operating life waiting for an event. Running the CPU at full speed during that idle time wastes power for no benefit. This report documents a small demonstration built on the ATmega328P that shows how to move the microcontroller into a low-power Power-down sleep state and wake it reliably on an external event, using a push-button wired to the INT0 pin as the trigger.

The system combines a status LED, a buzzer, and an I2C-driven character LCD (via a PCF8574 I/O expander) so that the transition between the awake and sleeping states is visible and audible, not just measurable on a bench meter.

2. Hardware Configuration

The demonstration uses the following pin assignments on the ATmega328P:

- LED — Port B, Pin 0 (PB0), configured as digital output
- Buzzer — Port B, Pin 1 (PB1), configured as digital output
- Wake-up button — Port D, Pin 2 (PD2 / INT0), input with internal pull-up
- Character LCD — connected via I2C through a PCF8574 I/O expander

System clock: 16 MHz external/crystal source ($F_{\text{CPU}} = 16,000,000 \text{ Hz}$).

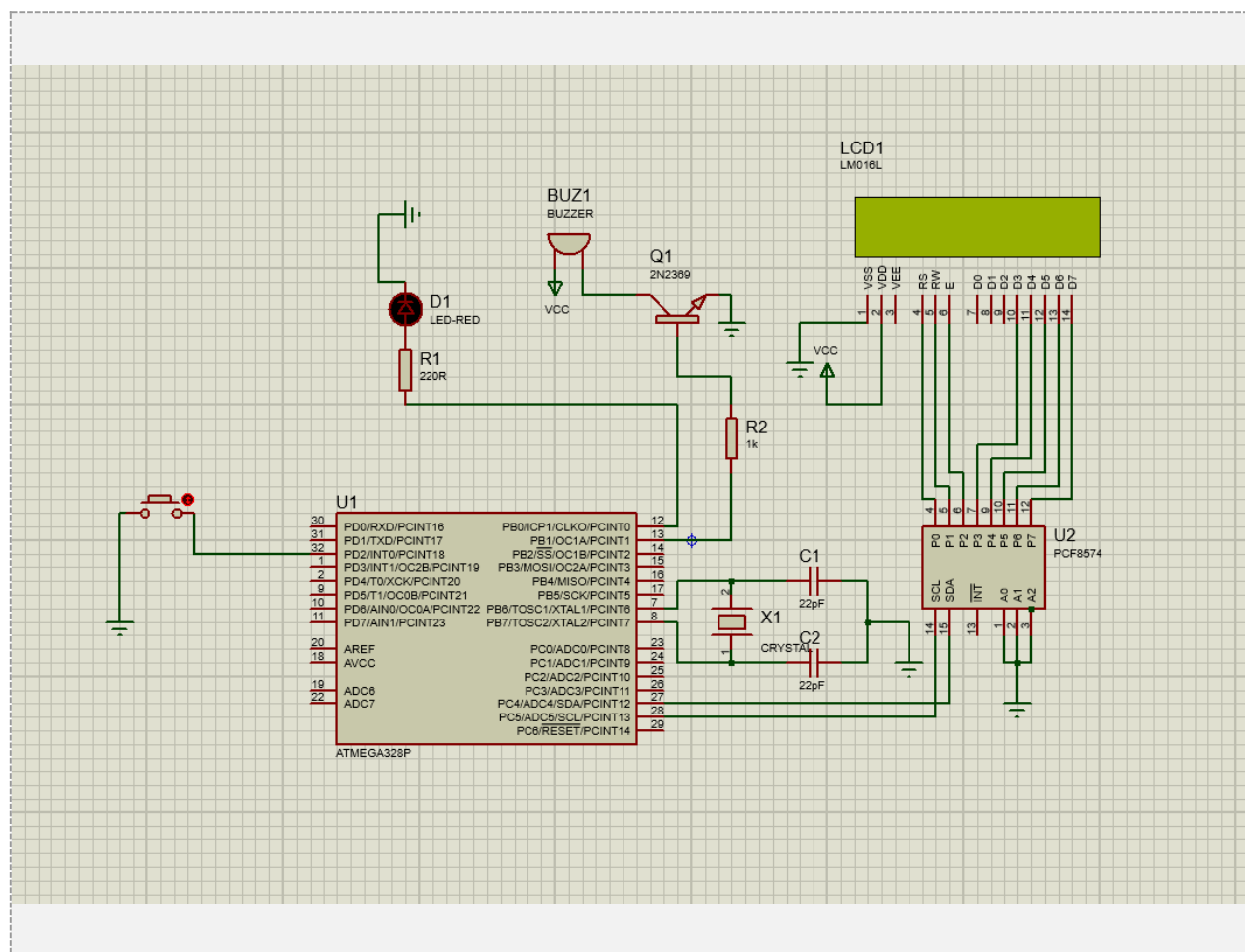


Figure 1. Hardware setup

3. Software Design

3.1 Wake-up Interrupt Configuration

INT0_Init() prepares PD2 as an input with its internal pull-up resistor enabled, then configures the interrupt sense control bits (ISC01:ISC00 = 00) so that INT0 triggers on a falling edge, i.e. when the button is pressed and pulls the pin low. The interrupt is enabled in EIMSK and global interrupts are enabled with sei(). The corresponding ISR(INT0_vect) does the minimum necessary work: it sets a volatile flag, button_flag, so the main loop can react after control returns from sleep.

3.2 Entering Power-Down Sleep

go_to_sleep() disables the ADC first (ADCSRA), since a running ADC prevents the deepest sleep modes from reaching their full power savings. It then writes SMCR to select Power-down mode (SM1=1, SM0=0) and sets the Sleep Enable (SE) bit, before executing the sleep instruction directly in inline assembly. In Power-down mode the CPU, system clock, and I/O and timer clocks are all stopped; only external interrupts and the watchdog timer (if enabled) remain active. Execution resumes at the instruction immediately after sleep once INT0 fires, and the function clears SE on the way out so a stray SMCR value cannot cause an unintended re-entry into sleep.

3.3 Main Program Flow

The main loop checks button_flag on each pass. When set, it lights the LED and buzzer, prints a status message to the LCD, briefly holds that state, then powers down the LED, buzzer, and LCD before calling go_to_sleep(). The MCU then idles with everything off until the button is pressed again, at which point the ISR sets the flag, execution resumes, and the loop prints a short wake-up message before repeating. The flow is summarized below.

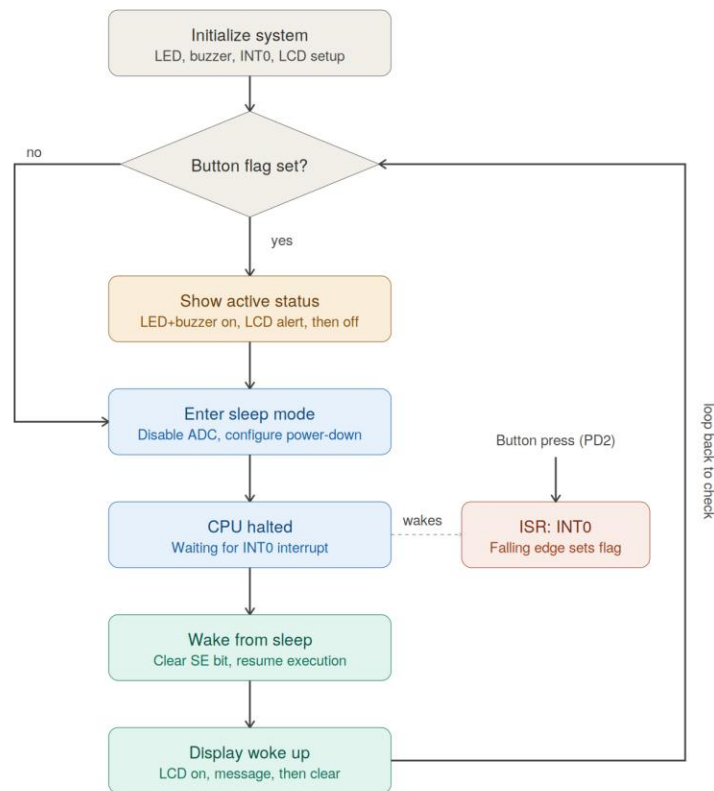


Figure 2. Main loop and sleep/wake control flow.

4. Power Management Considerations

- ADC is disabled before sleep , an active ADC blocks the lowest-power sleep modes.
- LED, buzzer, and LCD are explicitly switched off before sleep, since a peripheral left driven high still draws current even while the CPU is halted.
- The internal pull-up on PD2 avoids the need for an external resistor and keeps the pin in a defined state during sleep.
- Power-down mode gives the largest power reduction of the ATmega328P sleep modes at the cost of the slower wake-up latency associated with restarting the oscillator.

5. Source Code

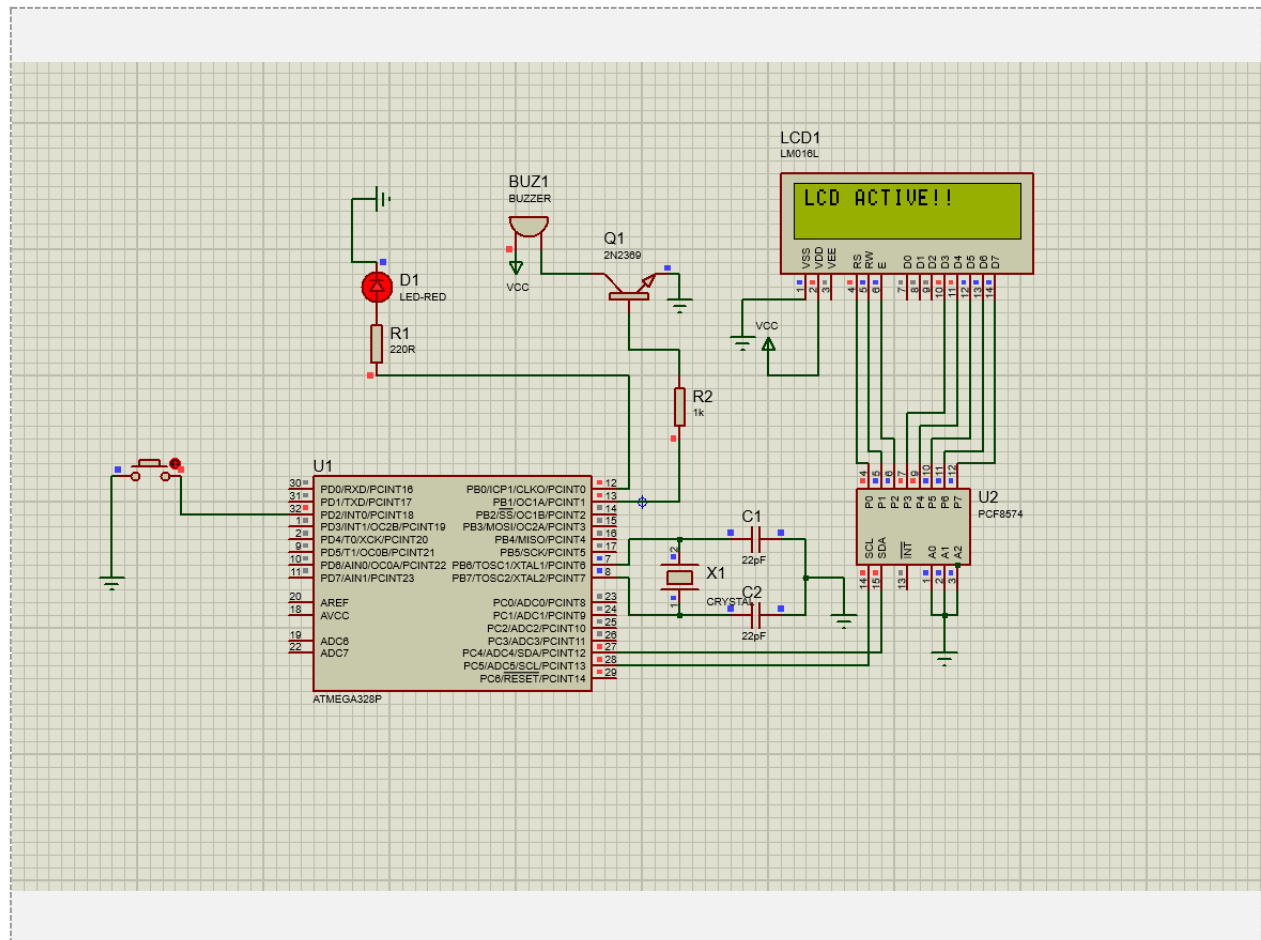
The source code for this task can be found on the Github repository accessed through this link:
https://github.com/eyobugr-4254-18-prog/insa_tasks

6. Testing and Results

The circuit was tested by pressing the button repeatedly and confirming that: the LED and buzzer activate only while awake, the LCD correctly reports “LCD ACTIVE!!”, “GOING TO SLEEP!!”, and “WOKE

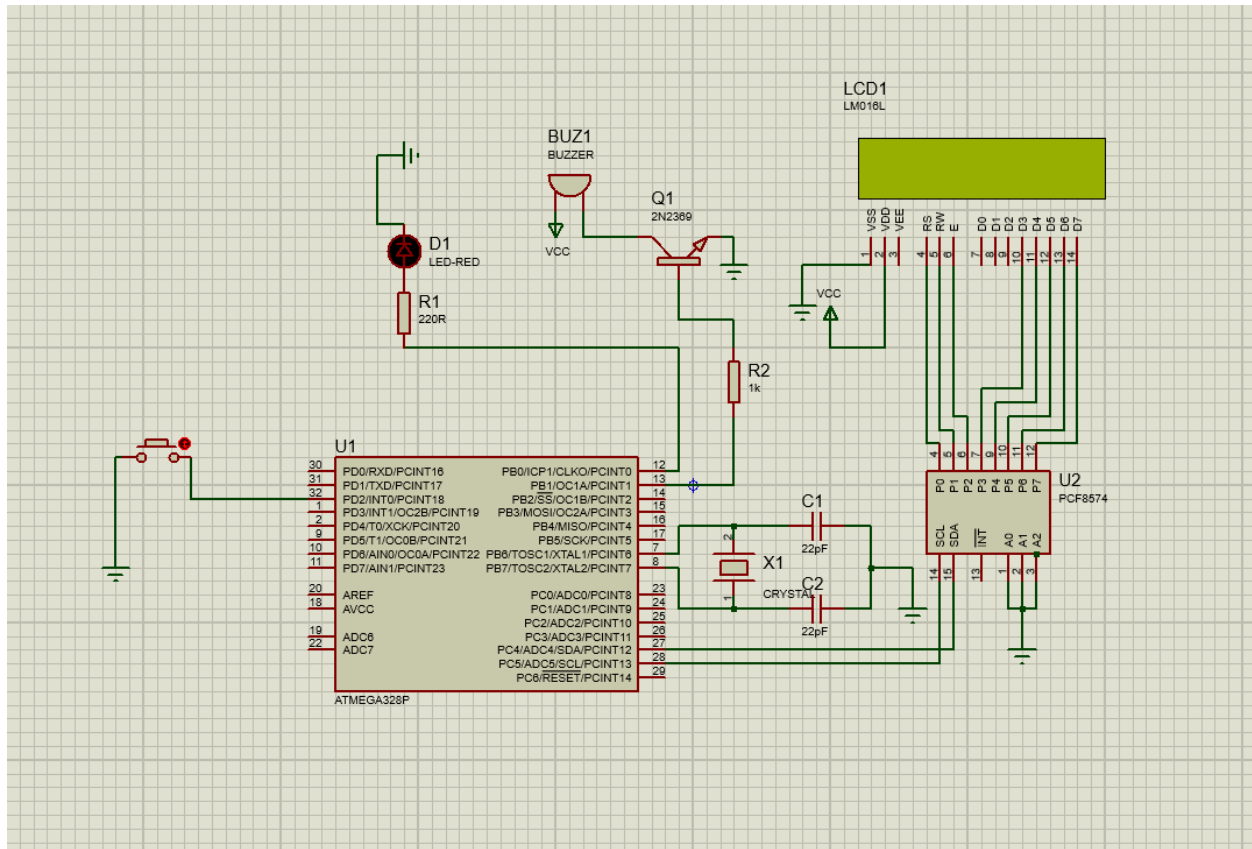
UP!!” at the expected points in the cycle, and the board draws visibly less current while in Power-down mode with all peripherals off.

Normal Mode



[illegible]

Sleep Mode



Wake-up Mode

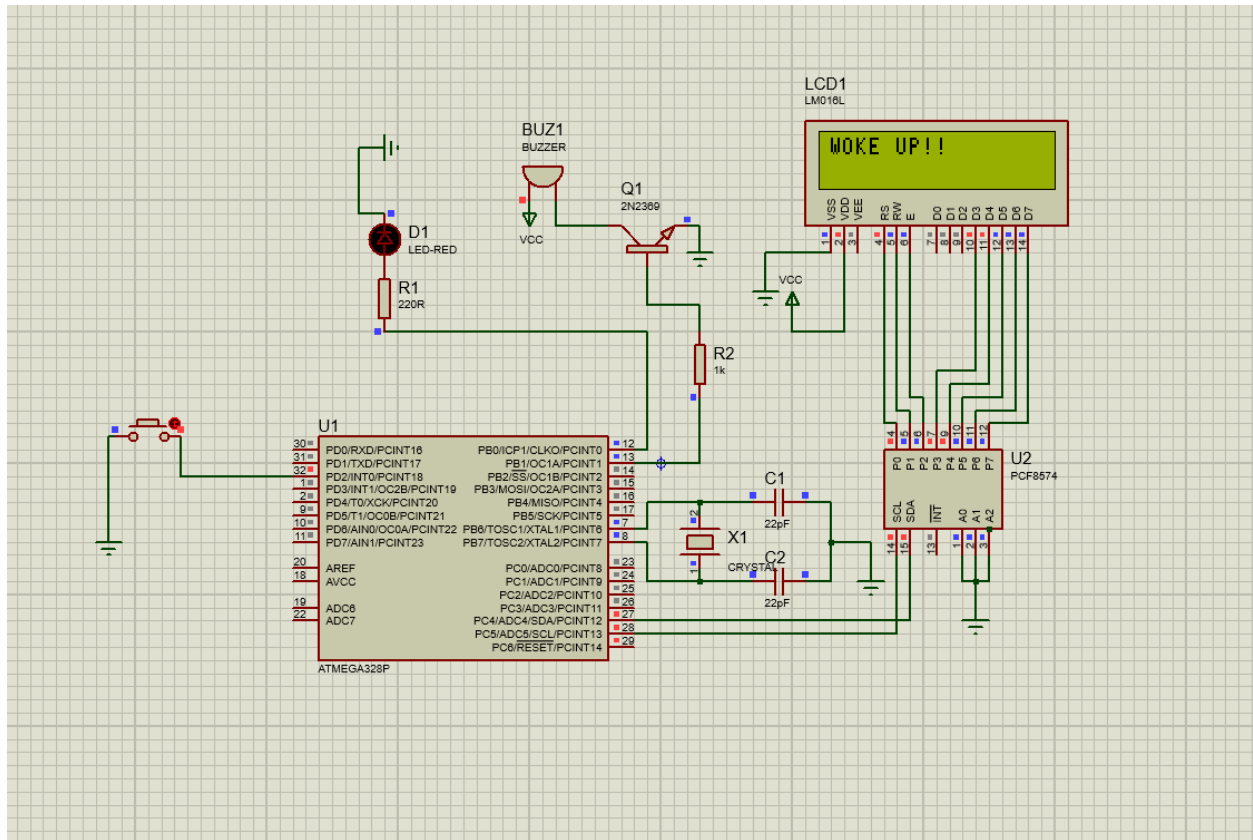


Figure 3. LCD output during an active/wake cycle.

7. Conclusion

This demonstration confirms a working, interrupt-driven sleep/wake cycle on the ATmega328P using Power-down mode and INT0. The approach, disable unused peripherals, sleep the CPU, and wake only on a meaningful external event, is a standard building block for battery-powered embedded designs and can be extended with the watchdog timer for periodic wake-ups or with additional interrupt sources for multi-button designs.